02_covered_indexes

Propietario	© Carlos Roberto
Etiquetas	

2.0 ENVIRONMENT CHECK

```
USE STN_Lab;
GO
-- ============================================================
==
-- 2.0 -- ENVIRONMENT CHECK
-- ============================================================
==
SELECT
    i.index_id,
    i.name                                          AS index_
name,
    i.type_desc                                     AS index_
type,
    i.is_primary_key,
    STRING_AGG(
        CASE WHEN ic.is_included_column = 0 THEN c.name END,
', ')
        WITHIN GROUP (ORDER BY ic.key_ordinal)      AS key_co
lumns,
    STRING_AGG(
        CASE WHEN ic.is_included_column = 1 THEN c.name END,
', ')
        WITHIN GROUP (ORDER BY ic.key_ordinal)      AS includ
ed_columns
FROM sys.indexes i
JOIN sys.index_columns ic
```

```
    ON i.object_id = ic.object_id
   AND i.index_id  = ic.index_id
JOIN sys.columns c
    ON ic.object_id = c.object_id
   AND ic.column_id = c.column_id
WHERE i.object_id = OBJECT_ID('dbo.Contribuyente')
GROUP BY i.index_id, i.name, i.type_desc, i.is_primary_key
ORDER BY i.index_id;
GO
```

Results | Messages

	index_id	index_name	index_type	is_primary_key	key_columns	included_columns
1	1	PK_Contribuyente	CLUSTERED	1	ContribuyenteID	NULL
2	2	IX_Contribuyente_NIT	NONCLUSTERED	0	NIT	NULL

Query executed successfully. | c13\lab_db (16.0 RTM) | carlos_roberto (51) | STN_Lab | 00:00:00 | Row: 1, Col: 1 | 2 rows

2.1 NonClustered INDEX (no INCLUDE)

IX_Contribuyente_NIT exists but covers only NIT.

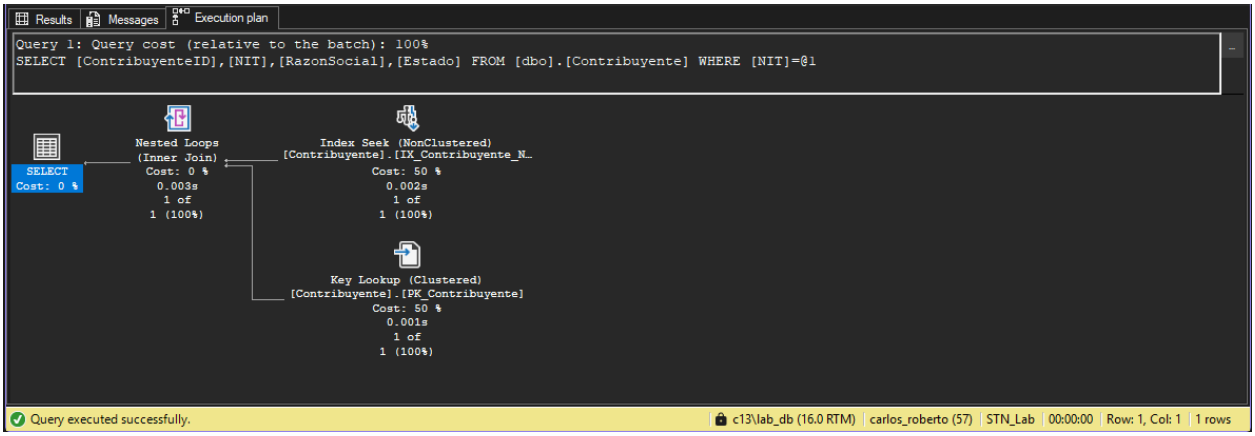
- Query requests RazonSocial and Estado -- both outside
- The index. Each matched row triggers a Key Lookup.
- Expected: Index Seek + Key Lookup (Nested Loops)

```
SET STATISTICS IO ON;
SET STATISTICS TIME ON;

SELECT
    ContribuyenteID,
    NIT,
    RazonSocial,
```

```
    Estado
FROM dbo.Contribuyente
WHERE NIT = '2010219';    -- replace with existing NIT

SET STATISTICS IO OFF;
SET STATISTICS TIME OFF;
GO
```

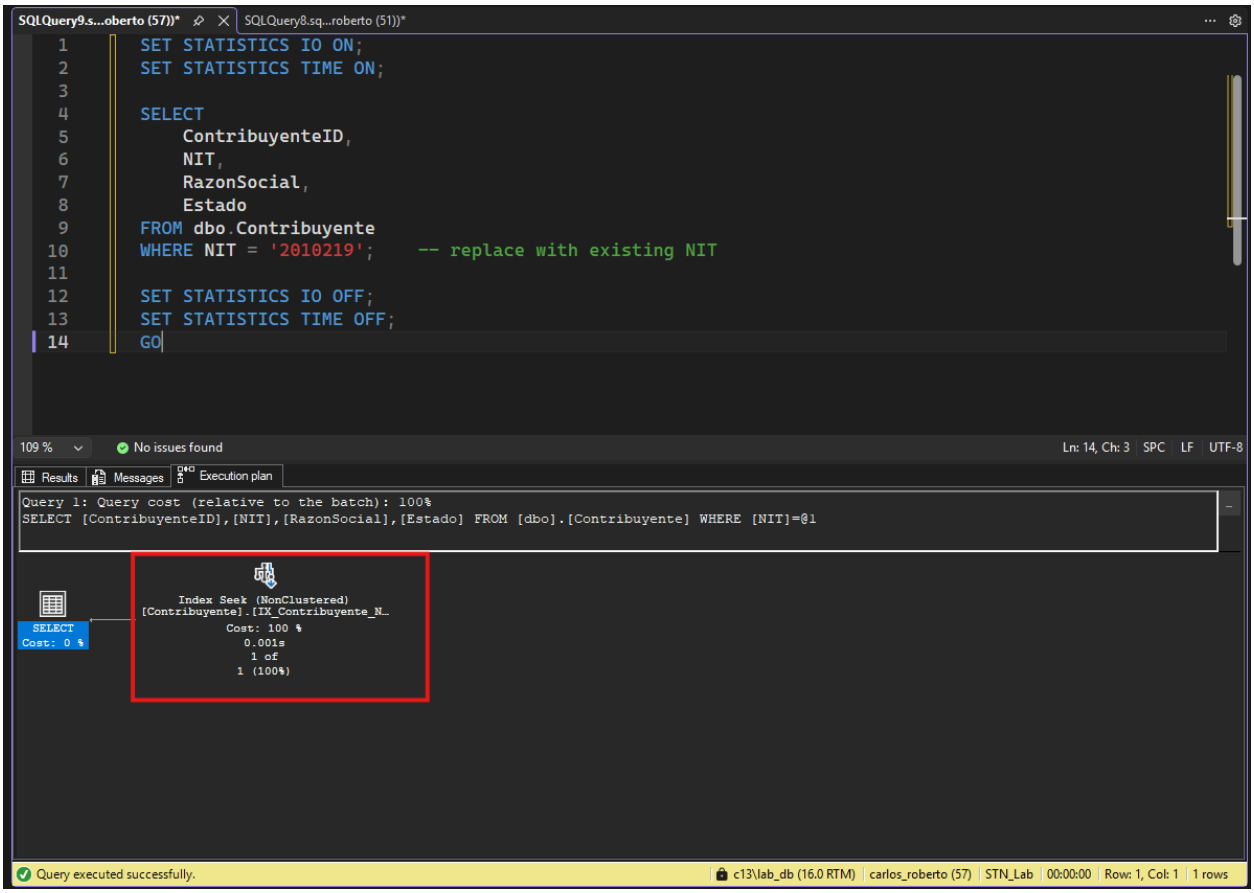


2.2 CREATE COVERING INDEX WITH INCLUDE

RazonSocial and Estado live only at the leaf level. They are not part of the sort key -- just passengers that. Eliminate the return trip to the clustered index

```
CREATE NONCLUSTERED INDEX IX_Contribuyente_NIT_Cobertor
    ON dbo.Contribuyente (NIT)
    INCLUDE (RazonSocial, Estado, ContribuyenteID);
GO

--repetir el script del ejemplo 2.1
```

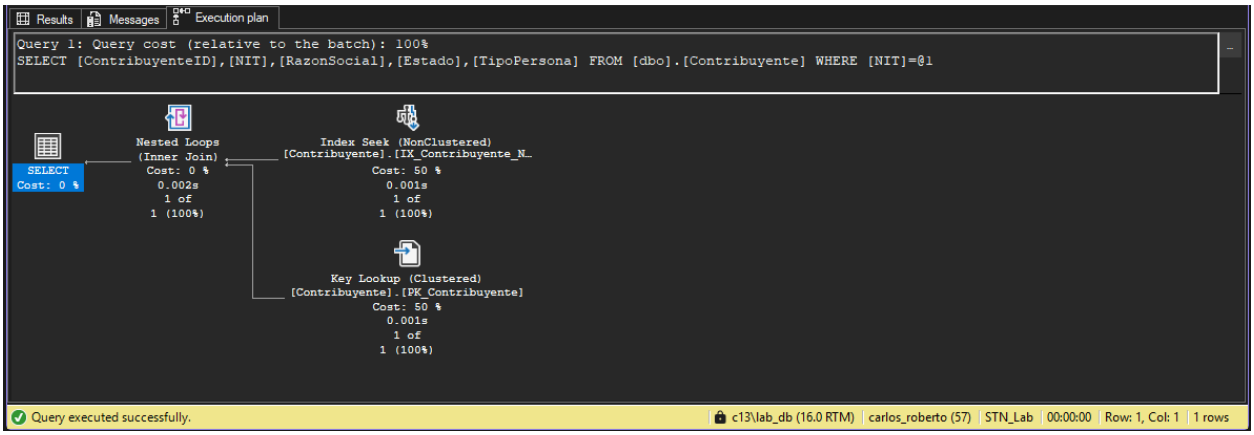


2.4 COVERAGE BREAKS WHEN COLUMN IS OUTSIDE INCLUDE

```
SET STATISTICS IO ON;

SELECT
    ContribuyenteID,
    NIT,
    RazonSocial,
    Estado,
    TipoPersona    -- not in INCLUDE
FROM dbo.Contribuyente
WHERE NIT = '2010219';
```

```
SET STATISTICS IO OFF;
GO
```



OBSERVACION 2.4

- Operator = Index Seek + Key Lookup (returned)
- Conclusion = coverage is query-specific, not table-wide

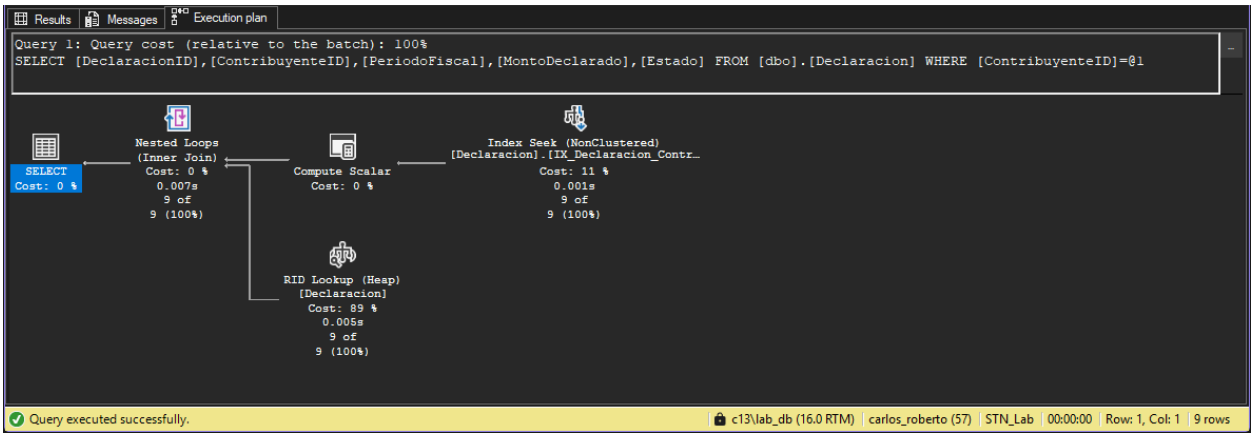
2.5 -- COVERING INDEX ON DECLARACION

2.5.1 Actual RID Lookup confirmed

```
SET STATISTICS IO ON;

SELECT
    DeclaracionID,
    ContribuyenteID,
    PeriodoFiscal,
    MontoDeclarado,
    Estado
FROM dbo.Declaracion
WHERE ContribuyenteID = 500;
```

```
SET STATISTICS IO OFF;
GO
```



2.5.2 Create covering index on Declaracion

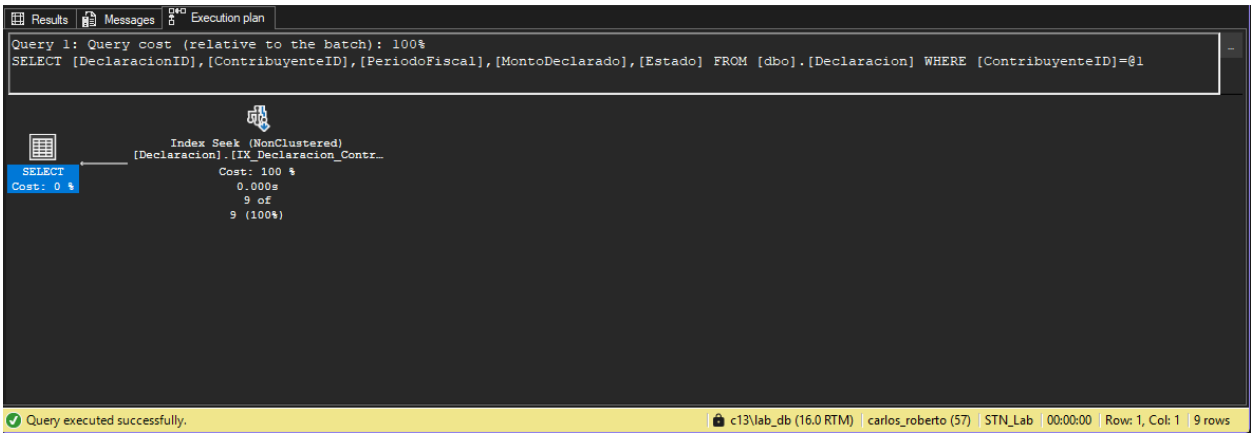
```
CREATE NONCLUSTERED INDEX IX_Declaracion_Contribuyente_Cubier
to
    ON dbo.Declaracion (ContribuyenteID)
    INCLUDE (PeriodoFiscal, MontoDeclarado, Estado, Declaraci
onID);
GO
```

2.5.3 Same query -- RID Lookup should disappear

```
SET STATISTICS IO ON;

SELECT
    DeclaracionID,
    ContribuyenteID,
    PeriodoFiscal,
    MontoDeclarado,
    Estado
FROM dbo.Declaracion
WHERE ContribuyenteID = 500;
```

```
SET STATISTICS IO OFF;
GO
```



Reads_2.5.3 = *3*

- Operator = Index Seek only , no RID Lookup
- Diferencia vs 2.5.1 =*9*

2.6 - INCLUDE VS KEY COLUMN

Las columnas INCLUDE solo existen en el nivel hoja.

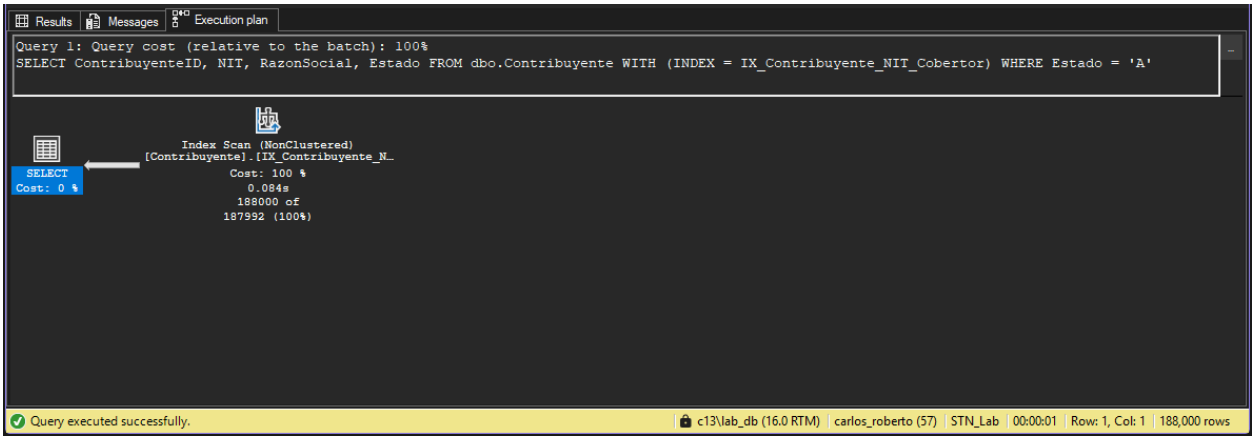
- No se pueden usar como predicados de filtro en WHERE.
- Si una columna necesita filtrar Y estar incluida, debe ser una columna clave en el índice, no solo INCLUDE.

```
SET STATISTICS IO ON;

SELECT
    ContribuyenteID,
    NIT,
    RazonSocial,
    Estado
FROM dbo.Contribuyente WITH (INDEX = IX_Contribuyente_NIT_Cob
ertor)
```

```
WHERE Estado = 'A';    -- Estado is INCLUDE, not a key column

SET STATISTICS IO OFF;
GO
```



2.7 SIZE COMPARISON: BARE vs COVERING INDEX

```
SELECT
    i.name                      AS index_name,
    s.used_page_count * 8       AS size_kb,
    s.row_count
FROM sys.dm_db_partition_stats s
JOIN sys.indexes i
    ON s.object_id = i.object_id
   AND s.index_id  = i.index_id
WHERE s.object_id IN (
    OBJECT_ID('dbo.Contribuyente'),
    OBJECT_ID('dbo.Declaracion')
)
  AND i.name IN (
    'IX_Contribuyente_NIT',
    'IX_Contribuyente_NIT_Cobertor',
    'IX_Declaracion_ContribuyenteID',
    'IX_Declaracion_Contribuyente_Cobertor'
)
```

```
ORDER BY s.object_id, i.name;
GO
```

Results | Messages | Execution plan

	index_name	size_kb	row_count
1	IX_Contribuyente_NIT	4312	200000
2	IX_Contribuyente_NIT_Cobertor	8760	200000
3	IX_Declaracion_Contribuyente_Cobertor	61712	1550000
4	IX_Declaracion_ContribuyenteID	68208	1550000

Query executed successfully. | c13\lab_db (16.0 RTM) | carlos_roberto (57) | STN_Lab | 00:00:00 | Row: 1, Col: 1 | 4 rows

La diferencia es el costo de storage de la cobertura.

2.8 CLEANUP

```
DROP INDEX IF EXISTS IX_Contribuyente_NIT_Cubierto        ON
dbo.Contribuyente;
DROP INDEX IF EXISTS IX_Declaracion_Contribuyente_Cubierto ON
dbo.Declaracion;
```